Temba Panoptic
Video Panoptic Segmentation via Vision Transformers with Temporal-Mamba modules

Alessandro Reali Federico Pezzoli Luca Ricci

Bocconi University
Milan, Italy
{alessandro.reali, federico.pezzoli, luca.ricci}@studbocconi.it

Abstract

Video panoptic segmentation is the task of jointly segmenting and tracking things and stuff across video frames. Current state-of-the-art methods apply image-based models such as Mask2Former independently per frame, or extend them with spatio-temporal attention. Both strategies struggle with temporal consistency: the same object can receive different IDs across frames, and its predicted mask can change shape or disappear from one frame to the next. We address this problem by inserting Temba blocks, Mamba-inspired temporal modules, inside Mask2Former, allowing the model to enrich multi-scale visual features with temporal context before class and mask prediction. We train and evaluate on JRDB-PanoTrack, a robot-centric dataset of crowded panoramic scenes, comparing our method against the unmodified Mask2Former and against a variant that uses extra transformer decoder layers instead of Temba blocks. We also study how the number of Temba blocks (3, 6 or 9, distributed across 3 adapters) affects performance. Results show that the one-block Temba configuration improves overall PQ, STQ, and VPQ over both baselines.

Code available at this GitHub repository.

1. Introduction

Video Panoptic Segmentation is a core task in video understanding, as it allows machines to correctly locate and classify objects (”things”) and background (”stuff”) in dynamic environments. One of the main challenges in this task is dealing with time dependencies: the segmentation should maintain its prediction stable across the whole length of the video. However, current state-of-the-art models are often either too weak to preserve long-range consistency or too expensive when applied across space and time. This leads to two primary failure modes: mask flickering, which happens when the model fails to keep prediction masks consistent across different frames, and identity drift, which happens when object tracking gets interrupted by occlusion or cluttering. Both these issues largely impact on many different video analytics tasks, from autonomous driving to robotic vision: in fact, temporal consistency is fundamental to any video understanding task that deals with long, untrimmed videos. For example, robots in crowded environments need to be able to distinguish each object correctly, even if it temporarily leaves the field of view; similarly, an assistance device for visually impaired patients should recognize and describe the environment correctly: both these applications heavily rely on the vision models' ability to maintain their prediction stable across multiple frames.

Addressing these problems, we propose Temba-Panoptic, a hybrid architecture that combines a strong panoptic segmentation backbone with Temba blocks used as a lightweight temporal feature-adaptation module. The central idea is to keep the spatial segmentation machinery unchanged while adapting Temba to inject temporal context into the multi-scale features used by the transformer decoder at multiple temporal scales. We initialize Mask2Former from a pretrained video checkpoint and initialize the Temba adapters from scratch. In the end-to-end experiments, both the original Mask2Former parameters and the newly introduced Temba parameters are fine-tuned on JRDB-PanoTrack. Indeed, our results show that all standard panoptic segmentation metrics benefit from the insertion of Temba blocks in the vision transformer pipeline, at a relatively low training cost.

2. Related Work

Video Panoptic Segmentation VPS unifies semantic and instance segmentation, requiring per-pixel class labels and consistent instance identities across video frames [8]. Two main paradigms exist: per-frame methods treat the video as a sequence of independently segmented images, then link predictions across frames with a tracking module; per-clip methods, instead, process a short window of frames jointly as a spatio-temporal volume and predict 3D masklets. State-

of-the-art image-based architectures such as Mask2Former [2] achieve strong per-frame segmentation results and have been extended to video [3]. These extensions inherit the per-frame model's limited mechanism for temporal reasoning. Our work follows the per-frame paradigm but augments it with lightweight temporal module, leaving the spatial backbone unchanged.

State Space Models and Mamba State Space Models (SSMs) [5] process sequences with linear complexity in the input length, an attractive property for long videos. Mamba [4] introduces selective SSMs, allowing content-dependent reasoning that matches Transformer accuracy on language tasks at lower cost. Mamba has since been adapted to vision (Vision Mamba [12], VMamba [10]), to time-series forecasting [7] and to action recognition in long untrimmed videos through MS-Temba [11] which introduces multi-scale temporal mamba blocks for temporal modeling. We adopt MS-Temba's multi-scale temporal block design and bring it, for the first time to our knowledge, to the video panoptic segmentation task.

Hybrid Mamba–Transformer architectures Some recent works combine Mamba with attention blocks rather than replacing one with the other. MambaVision [6] interleaves Mamba and self-attention layers within a vision backbone. Nemotron [1] integrates Mamba blocks into Transformer language models for efficient long-context reasoning. These works share our underlying intuition: Mamba and attention are complementary, and inserting Mamba into an existing Transformer pipeline can add new capabilities (long-range reasoning, efficiency) without discarding the strengths of attention. We apply this idea to video panoptic segmentation, inserting Temba blocks inside Mask2Former (between the pixel decoder and the transformer decoder) so that Mask2Former's spatial query mechanism is preserved while temporal consistency is modeled by the Mamba-inspired Temba adapters.

3. Methodology

The architecture we are using consists of two modules:

1. a **Segmentation module** relying on Mask2Former, which is fine-tuned on our dataset and performs VPS;
2. a **Temporal module**, which comprises Temba blocks and the necessary adapters, and is fully trained.

Our goal is to insert the Temporal module into the Segmentation module, enhancing temporal consistency across frames.

3.1. Mask2Former architecture

Mask2Former [2] is the baseline architecture we used as the starting point for the project. It is a transformer-based segmentation model designed for mask classification with the following structure:

1. At the beginning the input frames are processed by ResNet backbone, which extracts visual features at different spatial resolutions;
2. These multi-scale features are passed to a pixel decoder, which combines low-level spatial details with high-level semantic information to produce rich mask features;
3. Then, the transformer decoder uses a fixed set of learned queries to attend to these features and generate object-level or region-level representations
4. Finally, each query is converted into a class prediction and a corresponding segmentation mask.

In the context of VPS, Mask2Former provides strong per-frame segmentation capabilities, but by itself it has limited temporal awareness because it mainly processes frames independently. This means that it can produce accurate masks in individual frames while still suffering from temporal problems such as mask flickering or identity drift across consecutive frames.

3.2. Temba Blocks structure

Temba blocks [11] are temporal processing modules designed to give the model temporal context over the evolution of the video. In particular, in our implementation we used a simplified Temba-inspired temporal adapter based on local and dilated depthwise temporal convolutions rather than the entire state-space module. This particular choice comes from the fact that the main objective of our work was investigating whether it is possible to increase the temporal consistency of video segmentation task by using Temba blocks structure. For this reason, we started implementing the simplified Temba module based on local and dilated temporal convolutions, which preserves the feature format expected by the Mask2Former transformer decoder and can be trained end-to-end with limited architectural changes. With this approach we preserve (i) the dual-pathway structure combining a local temporal module with a dilated temporal mixer, rather than a single homogeneous operator; (ii) the depthwise, channel-wise treatment of the temporal axis, which keeps the adapter lightweight and avoids cross-channel mixing inside the temporal operator; (iii) the multi-scale design, realized through three scale-specific adapters with different dilations; this retains the inductive bias of MS-Temba.

This choice was also influenced by the limited computational resources available: the simplified convolutional variant is lighter, easier to integrate, and less memory-demanding than the full SSM implementation.

Specifically, our Temba blocks consists of three parts: a local temporal module, a temporal feature mixing and a feed-forward network. The first captures short-range temporal patterns, such as small movements between nearby frames, using one-dimensional convolution over time. The temporal mixer extracts information from a long-range time

dependencies, such as actions and movements that stay constant for long durations of time: in this way, the model can preserve object identity and reduce frame-to-frame inconsistencies. At the end, information is further processed through feed-forward layers and passed to the following block or to the Mask2Former decoder, while residual connections keep the original information available, preventing the temporal module from completely overwriting the spatial information coming from the pixel decoder.

The scheme of a Temba block used in our project is shown in figure 2 in appendix A.2.

3.3. Temba Mask2Former architecture

We modified the baseline Mask2Former architecture by injecting temporal adapters inside the segmentation pipeline. Starting from the pipeline for VPS with standard Mask2Former architecture, the backbone extracts visual features from each input frame, and the pixel decoder still produces multi-scale feature maps used for mask prediction. The main modification is that, before that features produced by the pixel decoder are passed to the transformer decoder for the inference tasks, they are enriched with temporal information through the Temba modules. Hence, Temba blocks take the multi scale spatial features, process them across consecutive frames, and returns updated features that contain both the spatial segmentation information extracted before by the pixel decoder and temporal context extracted by the Temba blocks. In this way, for each temporal window, the decoded features from consecutive frames are arranged along the temporal dimension and passed through the Temba adapters. This allows the model to exchange information between neighboring frames before the transformer decoder produces the final class and mask predictions. We denote by K the number of TEMBA-inspired blocks inside each scale-specific adapter. Since we use three adapters, $K = 1, 2, 3$ corresponds to 3, 6, and 9 total blocks.

Specifically, this placement of the Temba adapter allows temporal modeling to be applied at a stage where features are already spatially aligned and semantically enriched, but not yet collapsed into object-query representations. Consequently, the Temba blocks can enhance dense feature consistency and temporal dependency modeling while preserving the original backbone and decoder roles. In fact, after the pixel decoder, features already contain richer semantic and spatial information than raw backbone outputs, while still preserving dense pixel-level structure before they are compressed or specialized by the transformer decoder. Therefore, applying Temba blocks here allows temporal and global dependencies to be modeled directly over segmentation-ready feature maps, improving consistency and contextual reasoning without interfering with low-level backbone feature extraction or the final query-based mask decoding.

This design allows us to study whether increasing temporal processing improves video panoptic segmentation or instead degrades the spatial features passed to the transformer decoder.

The architecture scheme of Temba Mask2Former is shown in figure 3 in Appendix A.2.

3.4. Loss function

The training objective follows the standard Mask2Former [2] loss formulation:

$$\mathcal{L} = 2 \cdot \mathcal{L}_{CE} + 5 \cdot \mathcal{L}_{MASK} + 5 \cdot \mathcal{L}_{DICE} \tag{1}$$

where

1. $\mathcal{L}_{CE}$ is the balanced cross entropy function supervising the classification of each predicted query;
2. $\mathcal{L}_{MASK}$ supervises the pixel-level accuracy of the predicted masks;
3. $\mathcal{L}_{DICE}$ improves the region-level overlap between predicted and ground-truth masks, especially in the presence of class imbalance between foreground and background pixels.

This particular weight distribution allows the model to focus both on the labeling and on the segmentation tasks, giving a particular focus on the latter, as suggested by Cheng et al. in "Masked-attention mask transformer for universal image segmentation"[2] in section 4.1.

4. Dataset

As dataset we chose JRDB-PanoTrack [9] a robotic perception dataset built as an extension of JRDB for crowded, human-centric environments. It contains indoor and outdoor scenes captured by a mobile robot with synchronized 2D and 3D modalities: it consists of 4,000 360-degree panoramic frames sampled at 1 Hz from 54 videos, alongside 4,000 point clouds for 3D understanding. For our model, we used the 2D modality only, as it adapted best to the existing Mask2former backbone.

The dataset targets spatial and temporal scene understanding for robotics in settings where people, objects, walls and background regions coexist densely. Moreover, it supports both 2D panoptic segmentation, where models segment all thing and stuff regions in an image, and 2D panoptic tracking, where thing instances are tracked across video frames. These characteristics make the dataset particularly fit for our task, as they allow us to measure the model's performance in presence of cluttering and occlusion.

The annotations include 428K 2D panoptic masks, 27K tracking labels, and 7.3B annotated pixels across 72 object classes: 61 "thing" classes such as pedestrians, cars, chairs, tables, laptops, and bottles, plus 11 "stuff" classes such as sky, ground, and walls. The dataset also introduces multi-label panoptic annotation for cases like objects seen through

glass or hanging on walls, where a pixel may correspond to both a foreground surface and a visible object behind it: this is a notable feature, as it allows the dataset to represent complex real-world scenes more faithfully than standard single-label 2D segmentation datasets.

The distribution of the labels is not uniform and biased toward some categories: the class of "Pedestrian" is the most common one with over 10000 labels, while other classes have very few labels. In figure 1 of Appendix A.1 there is the distribution of the first 30 "thing" classes using logarithmic scale for better visualization.

These characteristics make JRDB-PanoTrack particularly challenging. The panoramic format introduces strong geometric variation, while crowded scenes contain frequent occlusions, small objects, and visually similar instances.

However, due to the strict limitation on computational resources available, in particular in terms of memory, we had to cut part of the dataset and we restricted our project to half of the entire JRDB PanoTrack dataset (27 videos). In particular, we used 24 videos for training and 3 videos for testing.

5. Experiments and Results

As explained above in section 3, we used Temba Blocks to enrich the Multi Scale features of the Pixel Decoder with the temporal dimension and use them as input of the decoder to predict the segmentation masks and the labeling of each element.

5.1. Evaluation Metrics

The evaluation was based on three main metrics, Panoptic Quality (PQ), Segmentation and Tracking Quality (STQ), and Video Panoptic Quality (VPQ), together with the number of predicted tracks.

Panoptic Quality evaluates the quality of panoptic segmentation at the single-frame level and is defined as

$$PQ = PQ_{SQ} \cdot PQ_{RQ} = \frac{\sum_{(p,g)\in TP} IoU(p,g)}{|TP| + \frac{1}{2}|FP| + \frac{1}{2}|FN|}, \quad (2)$$

where predicted segment p is matched with a ground-truth segment g only if their intersection over union is above the matching threshold (0.5 in our implementation). The two components of PQ are Segmentation Quality and Recognition Quality, defined as

$$PQ_{SQ} = \frac{\sum_{(p,g)\in TP} IoU(p,g)}{|TP|}$$

$$PQ_{RQ} = \frac{|TP|}{|TP| + \frac{1}{2}|FP| + \frac{1}{2}|FN|}.$$

While PQ_{SQ} measures the average mask overlap for correctly matched segments, PQ_{RQ} measures how well the model detects and classifies the segments.

Segmentation and Tracking Quality evaluates the video-level performance by combining semantic segmentation accuracy and temporal association accuracy. It is defined as

$$STQ = \sqrt{STQ_{SQ} \cdot STQ_{AQ}}, \quad (3)$$

where STQ_{SQ} is the semantic segmentation component and STQ_{AQ} is the association component. The segmentation component is computed as a mean intersection over union over the semantic classes:

$$SQ_{STQ} = \frac{1}{|C|} \sum_{c\in C} \frac{TP_c}{TP_c + FP_c + FN_c},$$

where C is the set of semantic classes . The association component measures whether the same object is assigned a consistent identity across frames and can be written as

$$AQ = \frac{1}{|G|} \sum_{g\in G} \frac{1}{|g|} \sum_{p:p\cap g\neq\emptyset} |p \cap g| \cdot IoU(p,g).$$

Here, G is the set of ground-truth object tracks, g is a ground-truth tube, and p is a predicted tube.

Finally, Video Panoptic Quality extends PQ from individual frames to short video clips. Instead of matching masks independently frame by frame, VPQ matches predicted and ground-truth tubes over a temporal window of k frames and applies a PQ-like formulation.

Although VPQ-1 uses a temporal window of one frame, its value is not necessarily equal to PQ because it is computed through the VPQ evaluation pipeline with a clip-based matching and a video-style aggregation rules, while PQ is computed directly as a standard per-frame panoptic metric.

5.2. Implementation details

Detectron2. We implemented all models within the Detectron2/Mask2Former framework, which provides dataset registration, data loading, training utilities, configuration management, and evaluation support. Starting from the standard Mask2Former video pipeline, we modified the segmentation head by inserting Temba temporal adapters between the pixel decoder and the transformer decoder, while keeping the original backbone, pixel decoder, decoder structure, optimizer setup, and loss formulation.

General implementation. All models are initialized from the Mask2Former R50 video checkpoint pretrained on YouTube-VIS 2021. The models are trained with AdamW, using an initial learning rate of 10^{-4}, weight decay 0.05, and a multi-step learning-rate schedule with linear warmup over 10 iterations. The learning rate is decayed by a factor of 10 at iteration 5,500. A learning-rate multiplier of 0.1 is applied to the backbone, and gradient clipping is applied with maximum norm 0.01. Training is performed with batch size 1 on a single GPU.

Training details. During training, data augmentation is applied by the Mask2Former video data mapper. Each training clip contains 2 frames sampled within a temporal window of 20 frames. Clips are randomly resized by sampling the short side from $\{360, 480\}$, with the long side capped at 1,333 pixels, and random horizontal flipping is applied consistently to all frames in the clip. At inference, frames are resized with short side 360 and long side up to 1,333. Predictions are then mapped back to the original frame resolution before computing PQ, STQ, and VPQ.

Temba blocks details. Each Temba block operates on 256-dimensional features. The Temba module consists of three scale-specific adapters, one for each selected pixel-decoder feature scale. Each adapter contains K stacked Temba blocks, with $K \in \{1, 2, 3\}$ in our experiments. The adapters use local temporal dilations $[1, 1, 2]$ and dilated temporal mixer dilations $[1, 2, 3]$. Temba blocks are initialized from scratch, while the remaining network parameters are initialized from the pretrained Mask2Former checkpoint. We train the model end-to-end, updating both the original Mask2Former parameters and the newly inserted Temba adapters. No Temba-specific loss is introduced; the temporal modules are supervised only through the standard Mask2Former segmentation losses.

5.3. Preprocessing on dataset

The preprocessing of the dataset was mainly focused on adapting JRDB-PanoTrack to the Mask2Former/Detectron2 training pipeline. The dataset already provides 360-degree panoramic video frames, obtained by merging multiple camera views, together with 2D segmentation masks and temporal labels for objects across frames. Before training, the frames and annotations had to be organized into a format compatible with the model: each image was associated with its corresponding semantic labels, instance masks, and tracking identifiers, so that the network could learn both the segmentation of objects in individual frames and their temporal continuity across the video.

The annotations were also structured according to the required panoptic format, separating "thing" classes, such as people or vehicles, from "stuff" classes, such as background regions. The images were then loaded, resized, normalized, and grouped consistently during training, while keeping the temporal order of frames so that the Temba blocks could exploit information from neighboring frames. This preprocessing step was essential because the model needed clean frame-mask associations and consistent object identities over time in order to evaluate improvements in segmentation quality, tracking consistency, and video panoptic performance.

5.4. Experiments done

The experimental setup was designed to compare the original Mask2Former baseline with several architectural modifications, in order to understand whether the performance gains came from generic model capacity or from the temporal information introduced by the Temba modules.

First, we trained the baseline Mask2Former model on JRDB-PanoTrack without any temporal adapter, using it as the reference point for all comparisons. Then, we tested a deeper version of the same model by adding three extra transformer decoder layers: this experiment was important because it allowed us to verify whether simply increasing the depth and capacity of the decoder could improve the results. After that, we introduced the proposed Temba adapters into the architecture and evaluated three configurations, using one, two, and three Temba blocks inside each adapter. These experiments were used to study how the amount of temporal processing affects video panoptic segmentation performance.

5.5. Results

The results of our experiment are reported in table 1 of A.3. The baseline Mask2Former achieves relatively low absolute scores on JRDB-PanoTrack, confirming the difficulty of the dataset and the limitations of a frame-based segmentation model. Increasing the capacity of the baseline by adding three extra decoder layers does not improve significantly the results: PQ decreases from 3.57 to 3.00 and VPQ decreases from 3.46 to 3.33. This shows that adding depth to the transformer decoder alone is not enough to increase temporal consistency. The only metric that increases is the number of tracked object (from 85 to 96).

In contrast, adding a single Temba block inside each scale-specific adapter produces the best overall performance. In fact, with one Temba block, PQ increases from 3.57 to 4.58, STQ from 17.74 to 19.93, and VPQ from 3.46 to 3.90. These improvements indicate that the temporal information introduced by Temba helps the model produce higher-quality predictions in the video setting. However, the gains mainly come from improved segmentation quality rather than from a clear improvement in association quality, since the STQ_{AQ} component remains close to the baseline and slightly decreases for $K = 1$. The gains are especially visible in the segmentation quality components, such as PQ_{SQ} and STQ_{SQ}, suggesting that the temporal module mainly improves the quality and stability of the predicted masks. In table 2 in Appendix A.3 it is shown the multiplicative factor of the 1 Temba block model with respect to the baseline.

However, increasing the number of Temba blocks to two or three does not lead to further improvements. Although the two-block model reaches the highest PQ_{SQ} value, its overall PQ and VPQ decrease because the recognition and

tracking components become weaker. This suggests that too much temporal processing can over-smooth the feature representations removing useful spatial details necessary for precise segmentation and classification. Therefore, the best trade-off is obtained with a single Temba block, which adds enough temporal context while better preserving the spatial information needed for mask prediction.

5.5.1. Second experiment: Inspecting how the metrics change with number of iterations

The second experiment, reported in table 3 of appendix A.3, analyzes the behavior of the two Temba block model at different training iterations. The results show a non-monotonic trend: at the beginning, some metrics improve with additional training, then they decrease, and finally they partially recover at the last stage. This behavior is consistent with the hypothesis that stacking multiple Temba blocks can make optimization harder and may degrade the features passed to the decoder. The partial recovery observed at 30k iterations suggests that longer training may help the temporal modules learn more useful representations, but the trend is not sufficient to conclusively prove this explanation. Other factors, such as optimization instability, learning-rate scheduling, or the limited validation set size, may also contribute to the observed fluctuations. Therefore, this experiment should be interpreted as preliminary evidence that deeper Temba configurations require more careful training and regularization.

5.6. Qualitative and visual inspections

In appendix A.4 it is possible to see one frame segmented by all the different models tested. As it is possible to see, the visual inspections confirm that the 1 Temba bolck model give the best output: for example, it is the only one, together with the 3 Temba blocks model, that correctly segments and labels the two bicycles and the riders on the left of the image. With respect to the baseline and to the model with more decoder blocks, the models using Temba blocks correctly segments the two pedestrian in the center of the image and the models with 2 or 3 Temba block manage to correctly segment and classify one of the light poles in the background too.

6. Limitations and future directions

The main limitations of this work are related to computational resources, dataset size, and annotation diversity. The main computational limitation was GPU memory, which restricted the batch size and required processing validation videos separately: for this reason, as said before, we considered only half of the dataset.

Moreover, the dataset is limited in domain diversity and has a long-tailed label distribution across its 72 classes, with many categories appearing rarely. For this reason, the conclusions drawn from the experiments should be considered specific to this setting and may not directly transfer to domains such as sports videos, autonomous driving, indoor robotics, or medical video analysis. Another limitation is that the proposed model introduces temporal reasoning only through the architecture, without using explicit temporal consistency or tracking loss. As a result, the model can improve video-level segmentation quality, but its ability to preserve object identities over time remains limited.

A natural future direction is to replace the simplified Temba adapters presented in section 3.2, with the full MS-Temba block, including the selective state-space scan. Such extension could improve long-range temporal reasoning and identity preservation across longer video sequences, especially in scenes with occlusions or objects temporarily disappearing from view.

Similarly, another possible direction is to model temporal consistency with shorter and possibly adaptive temporal windows, using a Markov chain-style formulation in which predictions depend primarily on recent frames. This could strengthen local temporal consistency while reducing the risk of over-smoothing caused by repeated temporal convolutions over longer contexts.

Another limitation of our implementation is that Temba is trained on 2-frame clips. Therefore, the temporal adapters mainly model short-term temporal consistency rather than long-range dependencies. Future work should train and evaluate Temba with longer temporal windows or sliding-window inference to fully exploit the multi-scale temporal dilations.

Finally, the model should be fine-tuned and evaluated on larger and more diverse datasets, including different domains such as sports, self-driving cars, surveillance, and embodied AI. This would make it possible to better assess the robustness, scalability, and generalization capacity of the Temba-enhanced Mask2Former architecture.

7. Conclusions

In conclusion, we introduced Temba-Panoptic, a Mask2Former-based video panoptic segmentation model augmented with lightweight Temba temporal adapters. By inserting the adapters between the pixel decoder and the transformer decoder, the model enriches multi-scale features with temporal context before producing class and mask predictions. Experiments on JRDB-PanoTrack show that the $N = 1$ configuration improves PQ, STQ, and VPQ over both the Mask2Former baseline and a deeper decoder variant. However, increasing Temba depth does not further improve performance, suggesting that temporal modeling must be balanced with preservation of spatial detail. Overall, the results indicate that lightweight temporal adapters are a promising direction for improving video panoptic segmentation, especially when combined with stronger regularization and explicit temporal consistency losses.

References

[1] Blakeman et al. Nvidia nemotron 3: Efficient and open intelligence, 2025.

[2] Cheng et al. Masked-attention mask transformer for universal image segmentation, 2021.

[3] Cheng et al. Mask2former for video instance segmentation, 2021.

[4] Gu and Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2024.

[5] Gu et al. Efficiently modeling long sequences with structured state spaces, 2022.

[6] Hatamizadeh and Kautz. Mambavision: A hybrid mamba-transformer vision backbone, 2024.

[7] Karadaga et al. ms-mamba: Multi-scale mamba for time-series forecasting, 2026.

[8] Kim et al. Video panoptic segmentation, 2020.

[9] Le et al. Jrdb-panotrack: An open-world panoptic segmentation and tracking robotic dataset in crowded human environments, 2024.

[10] Liu et al. Vmamba: Visual state space model, 2024.

[11] Sinha et al. Ms-temba: Multi-scale temporal mamba for understanding long untrimmed videos / efficient temporal action detection, 2025.

[12] Zhu et al. Vision mamba: Efficient visual representation learning with bidirectional state space model, 2024.

A. Appendix

Graphs of architecture and tables of results

A.1. Plot of distribution of labels in JRDB PanoTrack

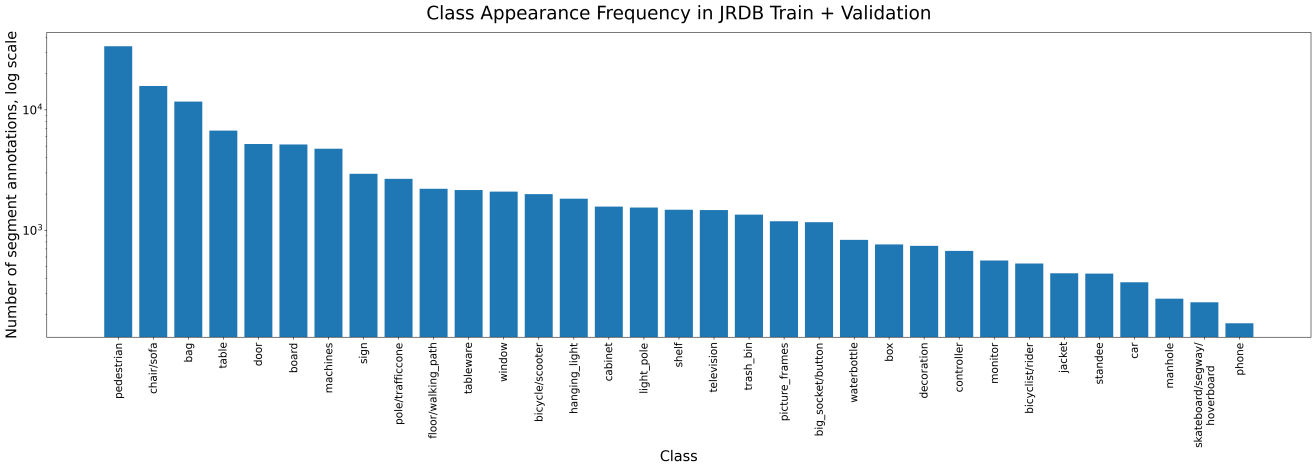

Figure 1. Distribution in logarithmic scale of first 30 labels in the training set

A.2. Plots of architectures

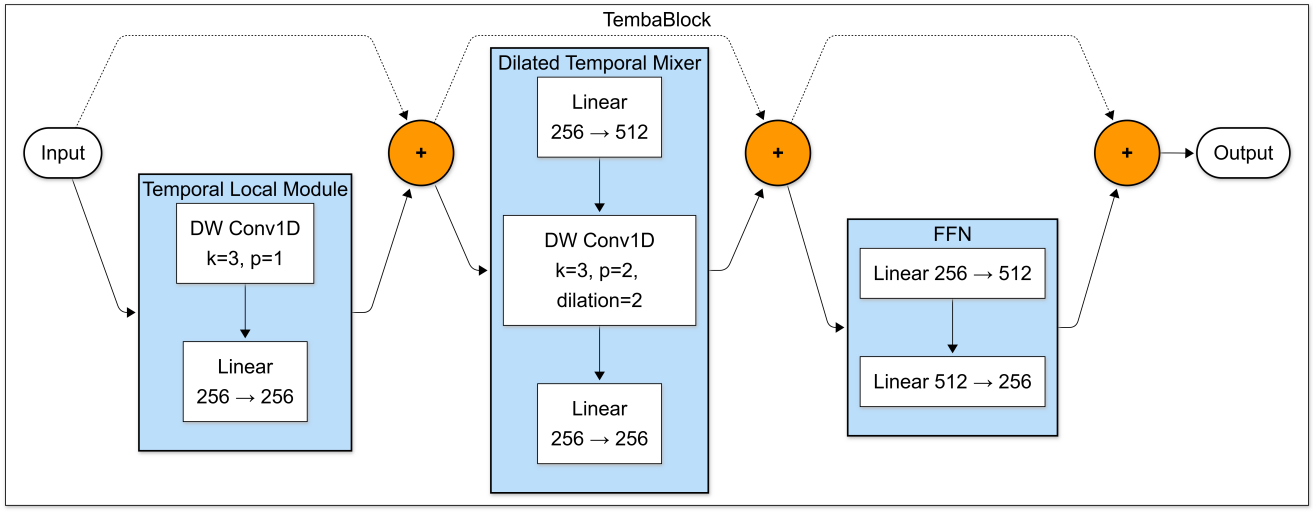

Figure 2. Basic scheme of a Temba block that will be inserted in the architecture.

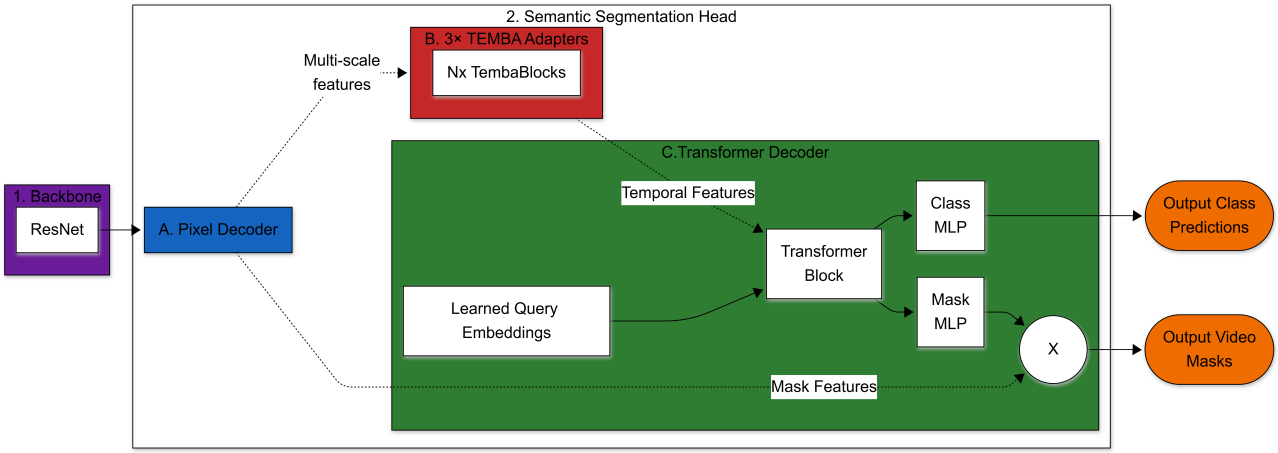

Figure 3. Scheme of the Temba Mask2Former architecture.

A.3. Tables of results

Method	PQ	SQ	RQ	STQ	SQ_{STQ}	AQ	VPQ	VPQ-1	VPQ-2	VPQ-4	VPQ-8	#tracks
Baseline (Mask2Former)	3.57	14.89	4.82	17.74	5.99	52.54	3.46	4.47	3.54	3.05	2.80	85
+ 3 decoder layers	3.00	14.92	4.05	17.81	6.10	**52.79**	3.33	4.27	3.47	2.94	2.66	**96**
Mask2Former + TEMBA												
$K = 1$	**4.58**	19.94	**6.40**	**19.93**	**7.63**	52.08	**3.90**	**4.93**	**4.04**	**3.55**	**3.10**	87
$K = 2$	3.46	**21.47**	4.58	17.15	5.62	52.36	3.21	4.23	3.34	2.78	2.50	94
$K = 3$	3.69	19.85	5.06	19.31	7.09	52.60	3.18	3.81	3.13	2.93	2.84	90

Table 1. Main results on JRDB-PanoTrack. We report panoptic quality (PQ), with its segmentation and recognition components PQ_{SQ} and PQ_{RQ}), segmentation tracking quality (STQ, with its segmentation and association components STQ_{SQ} and STQ_{AQ}), and video panoptic quality (VPQ) averaged over temporal windows of 1, 2, 4, and 8 frames, along with per-window VPQ values and the total number of tracks predicted and the number of objects tracked over the three testing videos. All models are trained for 30k iterations. **Bold** denotes the best value per column.

PQ	PQ_{SQ}	PQ_{RQ}	STQ	STQ_{SQ}	STQ_{AQ}	VPQ	VPQ-1	VPQ-2	VPQ-4	VPQ-8	#tracks
$\times 1.28$	$\times 1.34$	$\times 1.33$	$\times 1.23$	$\times 1.27$	$\times 0.99$	$\times 1.27$	$\times 1.10$	$\times 1.14$	$\times 1.16$	$\times 1.11$	$+2$

Table 2. Relative change of the K = 1 TEMBA model with respect to the Mask2Former baseline.

Iteration	PQ	SQ	RQ	STQ	SQ_{STQ}	AQ	VPQ	VPQ-1	VPQ-2	VPQ-4	VPQ-8	#tracks
6k	**4.56**	19.00	**4.65**	18.10	6.22	52.66	3.08	4.00	3.21	2.66	2.45	84
12k	3.42	21.04	4.52	**18.17**	**6.25**	52.53	**3.97**	3.13	**3.97**	**3.20**	2.61	88
18k	3.18	19.64	4.24	17.03	5.48	**52.93**	3.10	3.74	3.12	2.88	**2.69**	87
24k	3.13	18.41	4.21	17.15	5.58	52.71	2.86	3.61	3.04	2.50	2.29	93
30k	3.46	**21.47**	4.58	17.15	5.62	52.36	3.21	**4.23**	3.34	2.78	2.50	**94**

Table 3. Evolution of validation metrics across training iterations for the Mask2Former + TEMBA configuration with $K = 2$. **Bold** denotes the best value per column.

A.4. Example of segmentation output

As visual output for qualitative result we present the 24-th frame of the video ”meyer-green-2019-03-16_0”.

Figure 4. The example frame segmented with the baseline.

Figure 5. The example frame segmented with the more decoder blocks.

Figure 6. The example frame segmented with the 1 Temba block model.

Figure 7. The example frame segmented with the 2 Temba block model.

Figure 8. The example frame segmented with the 3 Temba block model.